BECE355L — Advanced Cloud Computing

Fall Semester 2026–27

Cloud-Native Parallel Computing Framework for Scalable Image Processing

Project Report

Python · OpenCV · Redis · FastAPI · Docker · Kubernetes

Submitted by

Name	Register No.
Prapti Kurkuri	24BVD0185
Vaanya Garg	24BVD0139
Aditya Manoj	24BVD0077
Arrnav Trisal	24BVD0064

Submitted to: **Faculty name**

25 October 2026

Abstract

Processing very large images (satellite scenes, medical scans, microscopy) on a single machine, one step after another, does not scale. Kubernetes can run many containers but has no built-in notion of splitting one large job into parallel tasks, while general-purpose distributed frameworks such as Spark or Ray are heavyweight for this purpose. This project builds a lightweight, container-first framework that divides a large image into tiles, schedules the tiles through a Redis task queue to a pool of parallel worker pods that process them with OpenCV, and aggregates the results back into the final image.

The framework supports the full workflow: image upload, partitioning with overlapping halos (so the stitched output is identical to whole-image processing), task scheduling, parallel processing, result aggregation and reconstruction. It is fault tolerant: every claimed tile carries a lease that the worker renews with heartbeats, and tiles held by a crashed worker are retried automatically. It scales dynamically: an autoscaler adjusts the number of worker pods to the length of the task queue. The system is packaged with Docker and deployed with Docker Compose or Kubernetes (Minikube).

Performance was evaluated on an 8192 × 8192 image (67 megapixels). For the CPU-heavy bilateral filter the tiled parallel engine reached a **4.04× speedup with 8 workers** and **4.19× with 16** over sequential execution, with output identical to the sequential result. Through the complete framework (HTTP upload, Redis, workers, aggregation) the parallel phase sped up **4.27**× with 8 workers and end-to-end time improved **1.40**×, the gap being explained by the serial upload/partition and aggregation stages (Amdahl's law). Lightweight operations such as Canny are faster sequentially, because moving the data costs more than the computation saved. A deliberately killed worker was recovered automatically with no loss of output.

Keywords: cloud-native, parallel computing, Kubernetes, Redis, task queue, image processing, OpenCV, fault tolerance, autoscaling, speedup.

Contents

1. Introduction
2. Objectives
3. Literature Review
4. System Architecture
5. Tools and Technology Stack
6. Design and Implementation
7. Results: the System in Operation
8. Performance Evaluation
9. Testing
10. Deployment on Kubernetes and Mapping to AWS
11. Limitations and Future Work
12. Conclusion

References

Appendix A: Project Structure

Appendix B: Configuration and API Reference

1. Introduction

1.1 Background

Image sizes keep growing: a single satellite scene or whole-slide pathology image can have hundreds of megapixels. Many image operations (filtering, edge detection, denoising) are local: the value of an output pixel depends only on a small neighbourhood of input pixels. Such work is naturally data-parallel: the image can be cut into pieces, the pieces processed independently on different machines or cores, and the outputs reassembled. Cloud platforms provide exactly the elastic compute this needs, and containers plus Kubernetes make it practical to run many identical workers and replace them when they fail.

1.2 Problem Statement

1. **Sequential execution does not scale.** Large computational jobs take too long when run task-by-task on a single machine.
2. **Kubernetes stops at orchestration.** It manages containers well but has no built-in concept of splitting one large job into parallel tasks, tracking them and combining the results.
3. **Existing frameworks are heavyweight.** Spark, Ray and Celery solve distributed compute but are not container-first and carry significant setup overhead for a focused workload like tile-based image processing.

1.3 Motivation

The aim is a lightweight, container-first framework that combines automatic task splitting with cloud-native elasticity and fault recovery, filling the gap between plain Kubernetes and heavy distributed-compute frameworks. The framework should be small enough to understand completely, yet show the core ideas of distributed systems: partitioning, scheduling, at-least-once delivery with idempotent completion, leases and heartbeats, autoscaling, and the limits of parallel speedup.

2. Objectives

Table 1: Project objectives and outcomes

#	Objective	Outcome	Where
1	Design a cloud-native architecture that splits a large job into independent parallel tasks	Achieved: image → halo tiles → one Redis task per tile	§4, §6.1–6.2
2	Containerise and orchestrate the pipeline (Docker, Kubernetes/Minikube)	Achieved: one Docker image; Compose file; Kubernetes manifests with RBAC, probes, resource limits	§6.7, §10
3	Demonstrate measurable speedup: sequential vs 2/4/8 workers	Achieved: 4.04× at 8 workers (compute), 4.27× parallel phase end-	§8

		to-end	
4	Fault tolerance: retry a task automatically when its worker fails	Achieved: leases + heartbeats + reaper; demonstrated by killing a worker	§6.5, §7.3
5	Elastic scaling of worker pods from task-queue length	Achieved: queue-length autoscaler (+ HPA / KEDA alternatives); 1 → 8 pods on a live Minikube cluster	§6.6, §10.2

3. Literature Review

[1] Towards an Adaptive Runtime System for Cloud-Native HPC (2026)

Proposes adaptive runtime techniques for cloud-native HPC, focusing on dynamic CPU/GPU resources, load balancing and cloud variability, using Charm++ for adaptive and resilient parallel computing.

Relevance: Motivates dynamic resizing of the worker pool and over-decomposition (many more tiles than workers) so that load balances automatically.

https://arxiv.org/abs/2603.14630

[2] An Elastic Job Scheduler for HPC Applications on the Cloud (SC25, 2025)

Introduces an elastic scheduler for HPC workloads on Kubernetes that scales resources according to cluster conditions to improve utilisation and reduce job response time.

Relevance: Our queue-length autoscaler is a simple instance of this idea: demand (pending tiles) drives the replica count of the worker Deployment.

https://ieeexplore.ieee.org/document/11358356/

[3] Serverless Data Pipeline Architecture Supporting Distributed Machine Learning on Fog Devices (2026)

Combines serverless computing, Kubernetes and distributed ML using OpenFaaS and KubeRay, and explores fault tolerance across edge, fog and cloud.

Relevance: Shares the queue-plus-stateless-workers design; our workers are stateless and any of them can take any tile, which is what makes retry and scaling simple.

https://www.sciencedirect.com/science/article/pii/S0140366426000952

[4] A Kubernetes-Based Scheme for Efficient Resource Allocation in Containerized Workflow (2025)

Proposes dynamic resource allocation for Kubernetes workflows that predicts future workload requirements, improving execution time and utilisation.

Relevance: Points to future work: predictive scaling from the size of submitted jobs instead of reacting to queue length.

https://www.sciencedirect.com/science/article/pii/S0167739X240066305

[5] Running Cloud-Native Workloads on HPC with High-Performance Kubernetes (2024)

Studies running cloud-native workloads directly on HPC systems by combining Kubernetes with HPC infrastructure, addressing the gap between the two worlds.

Relevance: Supports the premise of the project: Kubernetes is a viable substrate for parallel compute when a thin scheduling layer is added on top.

https://arxiv.org/abs/2409.16919

Compared with general frameworks (Apache Spark, Ray, Celery), this project trades generality for transparency: the whole scheduling layer is a few hundred lines on top of Redis primitives, every state transition is an atomic Lua script, and the system is container-native from the start.

4. System Architecture

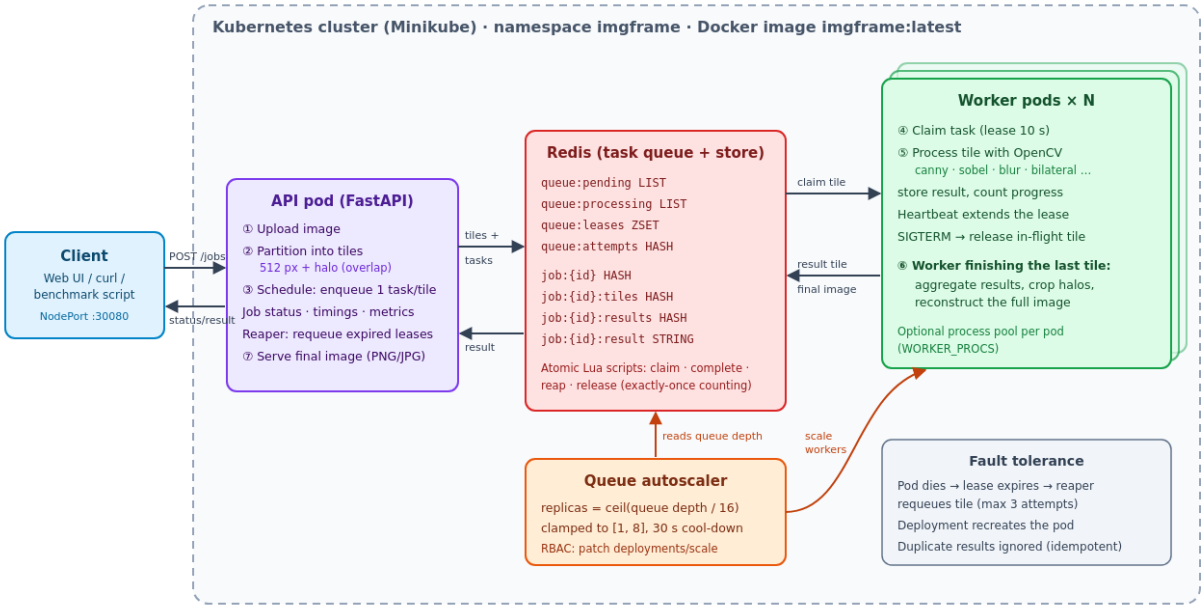


Figure 1: System architecture and data flow

The system has four kinds of components, all built from one Docker image:

- **API service (FastAPI).** Accepts uploads, partitions the image, enqueues tasks, reports job status, timings and cluster metrics, serves the final image and the web UI. It also runs the reaper that recovers tiles from failed workers.
- **Redis.** Acts as both the task queue (lists, a sorted set of leases) and the shared store for tiles, per-tile results, job metadata and the final image.
- **Worker pods.** Stateless processes that claim tiles, run the OpenCV operation and write results. The worker that completes a job's last tile performs aggregation and reconstruction.
- **Autoscaler.** Watches the queue depth and scales the worker Deployment through the Kubernetes API.

Data flow (numbers match Figure 1):

1. The client uploads an image with POST /jobs, choosing the operation and tile size.

2. The API decodes it and splits it into a grid of tiles, each with a halo of overlapping context pixels.
3. Tiles are written to Redis and one task per tile is pushed onto queue:pending.
4. Workers atomically move a task to queue:processing and record a lease.
5. Each worker processes its tile with OpenCV and stores the result. Progress is counted exactly once per tile.
6. The worker that completes the final tile crops the halos and stitches the full image.
7. The client polls GET /jobs/{id} and downloads the result from GET /jobs/{id}/result.

5. Tools and Technology Stack

Table 2: Technology stack

Layer	Technology	Role in the project
Language	Python 3.12	All services, tests and benchmarks
Image processing	OpenCV 4 (opencv-python-headless), NumPy	Canny, Sobel, Gaussian blur, bilateral filter, sharpen, grayscale; array slicing for tiles
API	FastAPI + Uvicorn	REST API, job tracking, web UI, Swagger documentation
Queue / store	Redis 7.2 + Lua scripts	Task queue with leases; tiles, results, metadata
Compute	concurrent.futures (ProcessPoolExecutor)	Optional process pool inside each worker; benchmark engine
Containers	Docker, Docker Compose	Single image for all roles; local multi-container runs
Orchestration	Kubernetes (Minikube)	Deployments, Services, probes, RBAC, scaling, self-healing
Scaling	Custom autoscaler (kubernetes Python client); HPA / KEDA manifests	Elastic worker count
Testing	pytest, fakeredis, httpx	26 automated tests
Measurement	psutil, matplotlib	CPU / memory sampling, charts

6. Design and Implementation

6.1 Image Upload and Partitioning

The uploaded image is decoded with OpenCV and divided into a grid of square tiles (512 × 512 px by default; edge tiles are smaller). A tile cannot be processed in isolation: a 15 × 15 blur near the tile border needs 7 pixels from the neighbouring tile, otherwise visible seams appear. Each tile therefore has a **core** region (the pixels it owns in the output) and a **halo** region, the core expanded by halo pixels on every side and clipped to the image.

Workers process the halo region; reconstruction keeps only the core. The halo is declared by each operation:

Table 3: Operations and their halo sizes

Operation	Kernel / method	Halo (px)	Tiled result vs whole image
gaussian_blur	15 × 15 Gaussian	7	identical
sobel	3 × 3 Sobel gradient magnitude	1	identical
sharpen	3 × 3 kernel	1	identical
bilateral	d = 9, σ = 75	4	identical
grayscale	colour conversion	0	identical
canny	5 × 5 Gaussian + Canny (50/150)	16	≥ 99.97 % of pixels identical (hysteresis can follow edges further than any finite halo)

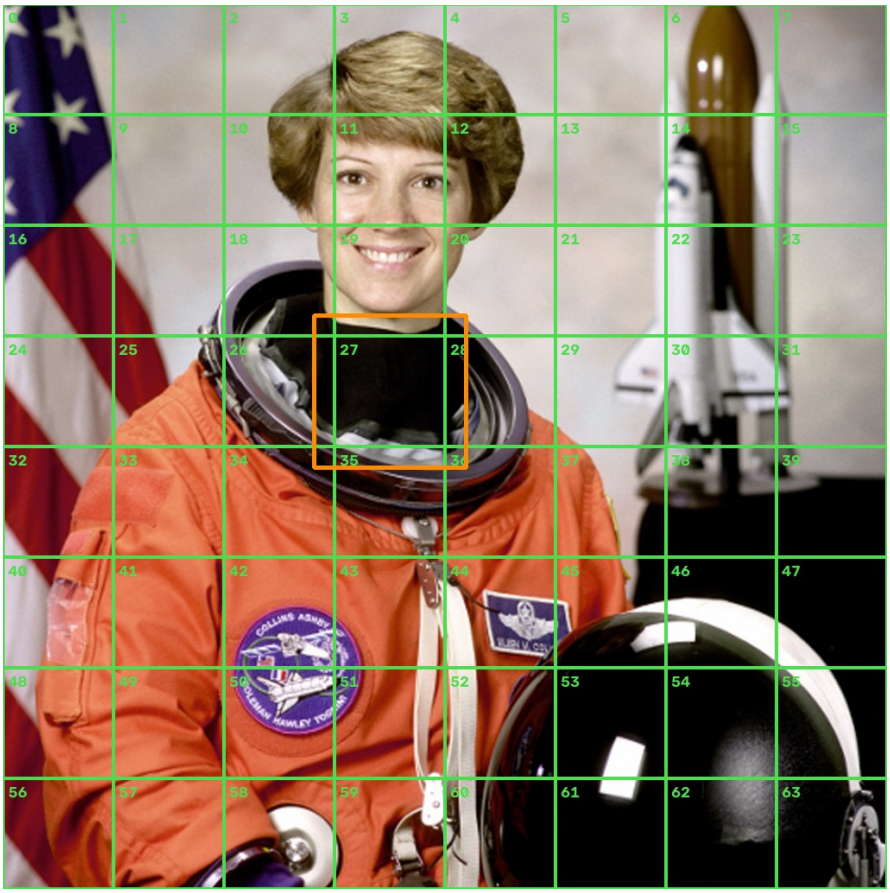


Figure 2: Partitioning a 2048 × 2048 image into 256 px tiles (green, numbered). The orange box shows the halo region of tile 27 (exaggerated ×3 for visibility).

```
def plan_tiles(height, width, tile_size, halo):
    for y0 in range(0, height, tile_size):
        for x0 in range(0, width, tile_size):
            y1, x1 = min(y0 + tile_size, height), min(x0 + tile_size, width)
            yield TileSpec(core=(y0, y1, x0, x1),
                           halo=(max(0, y0 - halo), min(height, y1 + halo),
                                 max(0, x0 - halo), min(width, x1 + halo)))
```

Tiles travel between the API, Redis and the workers in a compact raw format (a 16-byte header with height, width and channels followed by the pixels). Profiling showed that PNG-encoding the tiles made partitioning and aggregation several times slower, and these tiles never leave the system.

6.2 Task Scheduling: a Reliable Queue on Redis

All scheduling state lives in Redis:

Table 4: Redis data model

Key	Type	Purpose
imgframe:queue:pending	LIST	Task ids (<job>:<tile>) waiting for a worker
imgframe:queue:processing	LIST	Tasks currently claimed
imgframe:queue:leases	ZSET	Task → lease expiry time; heartbeats push it forward
imgframe:queue:attempts	HASH	Task → number of claims (for the retry limit)
imgframe:job:<id>	HASH	Status, operation, size, tile count, done counter, timings
imgframe:job:<id>:tiles / :specs	HASH	Input tile bytes / tile geometry
imgframe:job:<id>:results / :completed	HASH / SET	Output tile bytes / indices already counted
imgframe:job:<id>:result	STRING	Reconstructed final image

Every state transition is a **Lua script executed atomically inside Redis**, so concurrent workers can never claim the same task twice or lose one between two commands. Claiming a task moves it from pending to processing, records its lease and marks the job running, in one step:

```
local task = redis.call('LMOVE', KEYS[1], KEYS[2], 'LEFT', 'RIGHT')   -- pending -> processing
if not task then return false end
redis.call('ZADD', KEYS[3], now + lease_seconds, task)              -- lease
redis.call('HINCRBY', KEYS[4], task, 1)                          -- attempt count
if redis.call('HSETNX', job, 'started', now) == 1 then
  redis.call('HSET', job, 'status', 'running')
end
return task
```

Because there are many more tiles than workers (an 8192^2 image at 512 px gives 256 tiles), scheduling is naturally load balanced: a fast worker simply claims more tiles. This over-decomposition is the same idea used by adaptive HPC runtimes [1].

6.3 Parallel Processing

A worker runs a loop: claim a task, load the tile, run the OpenCV operation, store the result. A background thread renews the lease of every in-flight tile every few seconds (heartbeat). Parallelism exists at two levels: across **worker pods** (scaled by Kubernetes) and, optionally, across a **process pool inside each worker** (WORKER_PROCS, using

concurrent.futures.ProcessPoolExecutor). OpenCV is pinned to one internal thread per process (cv2.setNumThreads(1)) so that all parallelism comes from the framework and the sequential baseline is genuinely sequential.

6.4 Result Aggregation and Reconstruction

When a worker finishes a tile, a Lua script removes the task from the processing list and adds the tile index to the job's completed set. Only if the index was not already there does it store the result and increment the job's done counter. Completion is therefore **idempotent**: if a slow worker and a retry both finish the same tile, it is counted once. The script returns done and total; the worker that observes done == total (exactly one, because the increment is atomic) performs aggregation. It loads all result tiles, crops each tile's halo, writes the cores into a new array and stores the reconstructed image. Per-tile data is then deleted. Encoding the final image as PNG/JPEG happens on the first download and is cached, so the job completes as soon as the pixels are ready.

6.5 Fault Tolerance

The framework provides at-least-once processing with exactly-once accounting:

Table 5: Failure scenarios and how they are handled

Failure	Detection	Recovery
Worker pod crashes or is killed (SIGKILL, OOM, node loss)	Its leases stop being renewed and expire (10 s on Kubernetes)	The reaper (in the API and every worker) moves expired tasks back to the front of the pending queue; Kubernetes recreates the pod
Worker shut down gracefully (scale-down, rolling update: SIGTERM)	Signal handler	In-flight tiles are released immediately without counting an attempt
Tile processing raises an error	Exception in the worker	The lease is expired at once, so the tile is retried
A tile fails repeatedly (e.g. corrupt data)	Attempt counter reaches MAX_RETRIES (3)	The job is marked failed with an error message; remaining tiles are discarded
Retried tile also finished by the original worker	Index already in completed set	Duplicate result ignored
Transient Redis connection drop	redis-py connection error	Client retries with exponential back-off; aggregation is retried 3 times
Redis memory nearly full	Admission check before accepting a job	API answers HTTP 503 with Retry-After (back-pressure); clients retry; finished results expire after 15 min or are deleted with DELETE /jobs/{id}

For testing, a worker can be told to crash at random while holding a tile (CHAOS_CRASH_RATE), and the local launcher can SIGKILL a worker mid-job (--kill-worker).

6.6 Dynamic Scaling

The autoscaler runs as its own Deployment with a ServiceAccount allowed only to read and patch the scale of the worker Deployment. Every 3 seconds it computes:

```
desired = clamp( ceil(queue_depth / TASKS_PER_WORKER), MIN_WORKERS, MAX_WORKERS )
        = clamp( ceil(queue_depth / 16), 1, 8 )
```

Scale-up is applied immediately; scale-down waits until demand has been low for 30 seconds to avoid churning pods between jobs. Workers removed during scale-down receive SIGTERM and hand back their in-flight tiles. Two alternatives are included: a standard CPU-based HorizontalPodAutoscaler and a KEDA ScaledObject that scales on the Redis list length.

6.7 Containerisation and Orchestration

A single Docker image (python:3.12-slim with the dependencies, running as a non-root user) contains all roles; the container command selects API, worker or autoscaler. Docker Compose runs Redis, the API and a scalable number of workers for local multi-container testing. The Kubernetes manifests define:

Table 6: Kubernetes manifests

Manifest	Objects	Key settings
00-namespace.yaml	Namespace, ConfigMap	Shared configuration (Redis URL, tile size, lease)
10-redis.yaml	Deployment, Service	redis:7.2-alpine, readiness probe, 3 GB maxmemory
20-api.yaml	Deployment, NodePort Service	Port 30080, readiness and liveness probes on /healthz
30-worker.yaml	Deployment	CPU request 0.5 / limit 1 per pod, 30 s termination grace
40-autoscaler.yaml	ServiceAccount, Role, RoleBinding, Deployment	Least-privilege RBAC, scaling policy via environment
optional/hpa-cpu.yaml	HorizontalPodAutoscaler	Alternative: 70 % CPU target
optional/keda-scaledobject.yaml	ScaledObject	Alternative: Redis list length

6.8 REST API and Web Interface

Table 7: REST API

Method	Endpoint	Description
POST	/jobs	Upload an image (multipart: file, op, tile_size); returns the job
GET	/jobs	Recent jobs
GET	/jobs/{id}	Status, progress, retries and timings of each phase
GET	/jobs/{id}/result	Final image; ?format=png|jpg|webp, ?max_side=N to

		downscale
GET	/jobs/{id}/preview	Downscaled copy of the upload
DELETE	/jobs/{id}	Delete a job and free its memory
GET	/metrics	Queue depth, in-flight tasks, retries, per-worker statistics
GET	/ops	Available operations
GET	/docs	Interactive API documentation (Swagger UI)

The web interface (served at /) lets the user upload an image, follow progress tile by tile, compare the original and the result, and see the per-phase timings. It also shows live cluster metrics and the list of workers with the number of tiles each has processed.

7. Results: the System in Operation

All screenshots in this section were captured from the running system: Redis 7.2, the FastAPI service and four worker processes on the development machine, driven through the web UI by an automated browser (scripts/capture_screenshots.py).

7.1 Web Interface

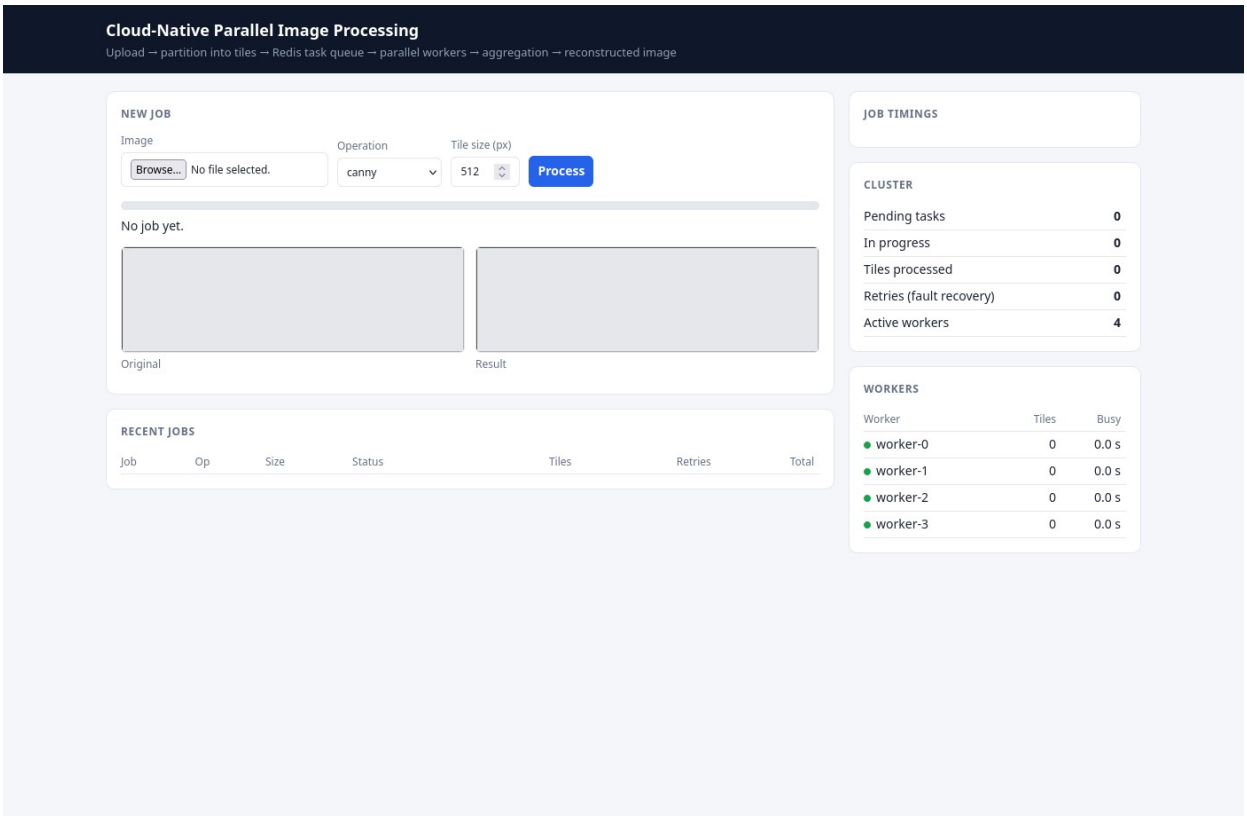


Figure 3: Web UI ready for a job; four workers registered

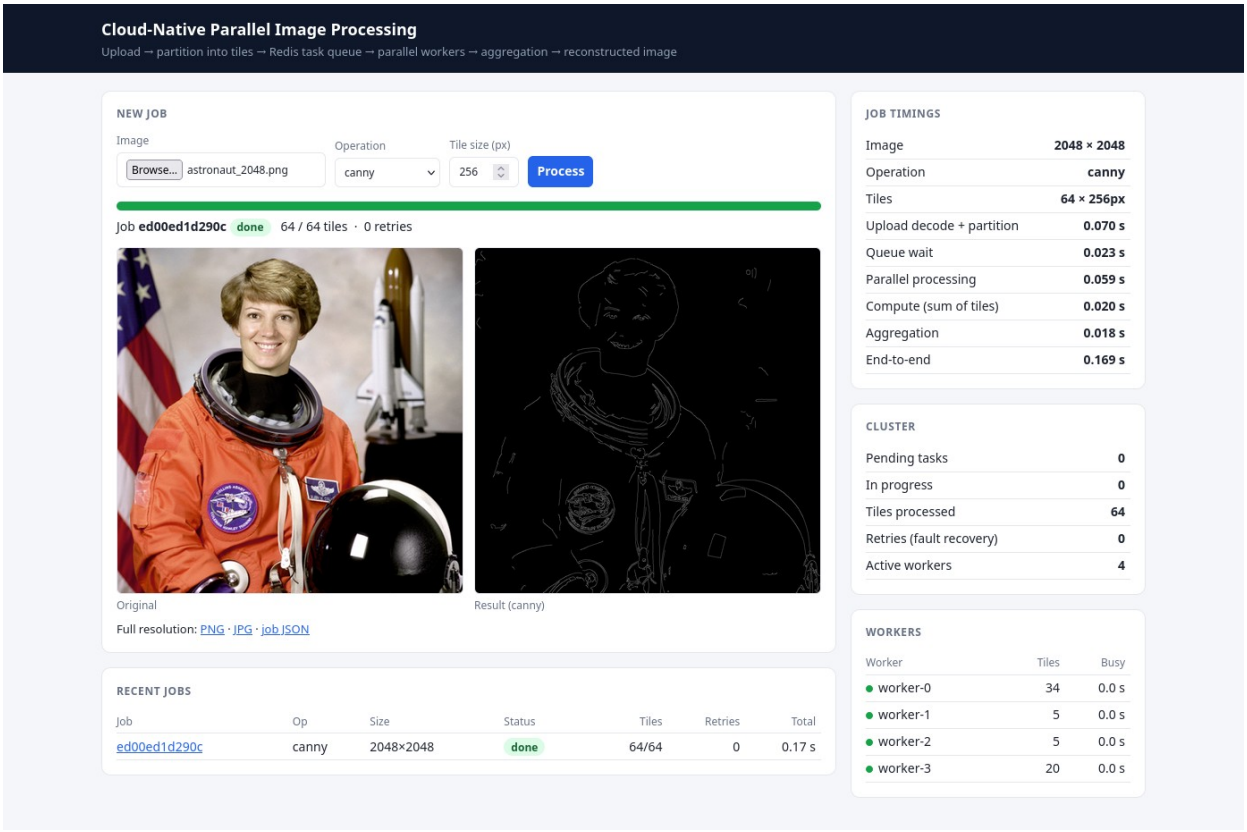


Figure 4: Canny edge detection on a 2048 × 2048 photograph split into 64 tiles of 256 px

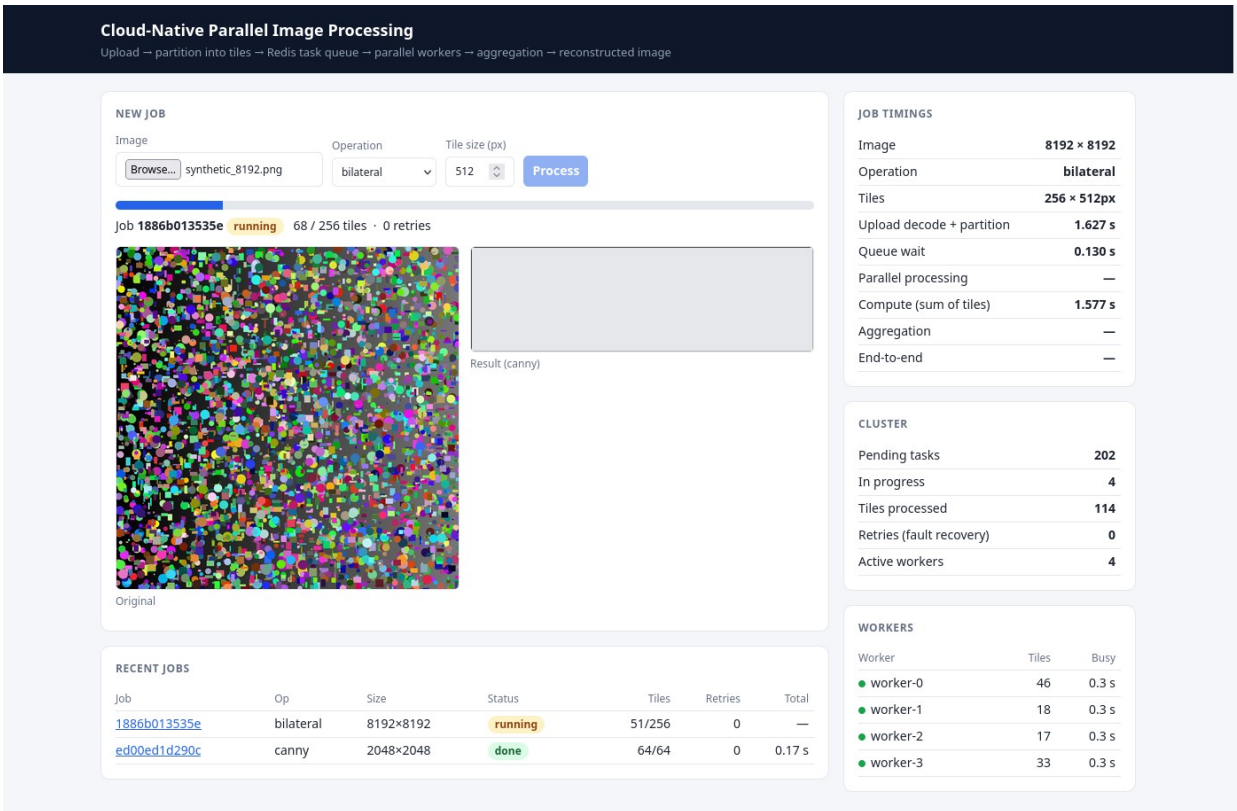


Figure 5: A 67-megapixel (8192 × 8192) bilateral-filter job in progress: tiles are being processed by four workers in parallel

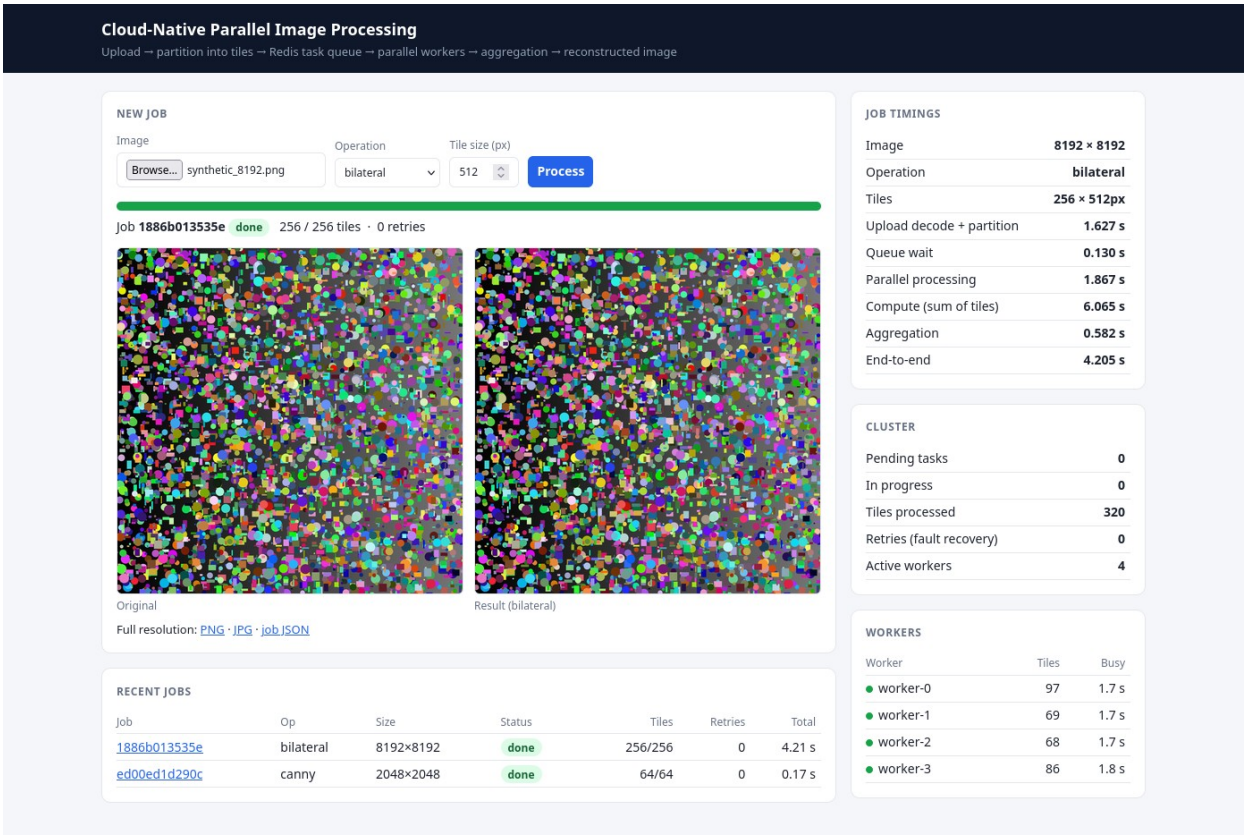


Figure 6: The same job completed: 256 tiles, timings for every phase, tiles shared evenly between the workers

7.2 Correctness of Tiled Processing

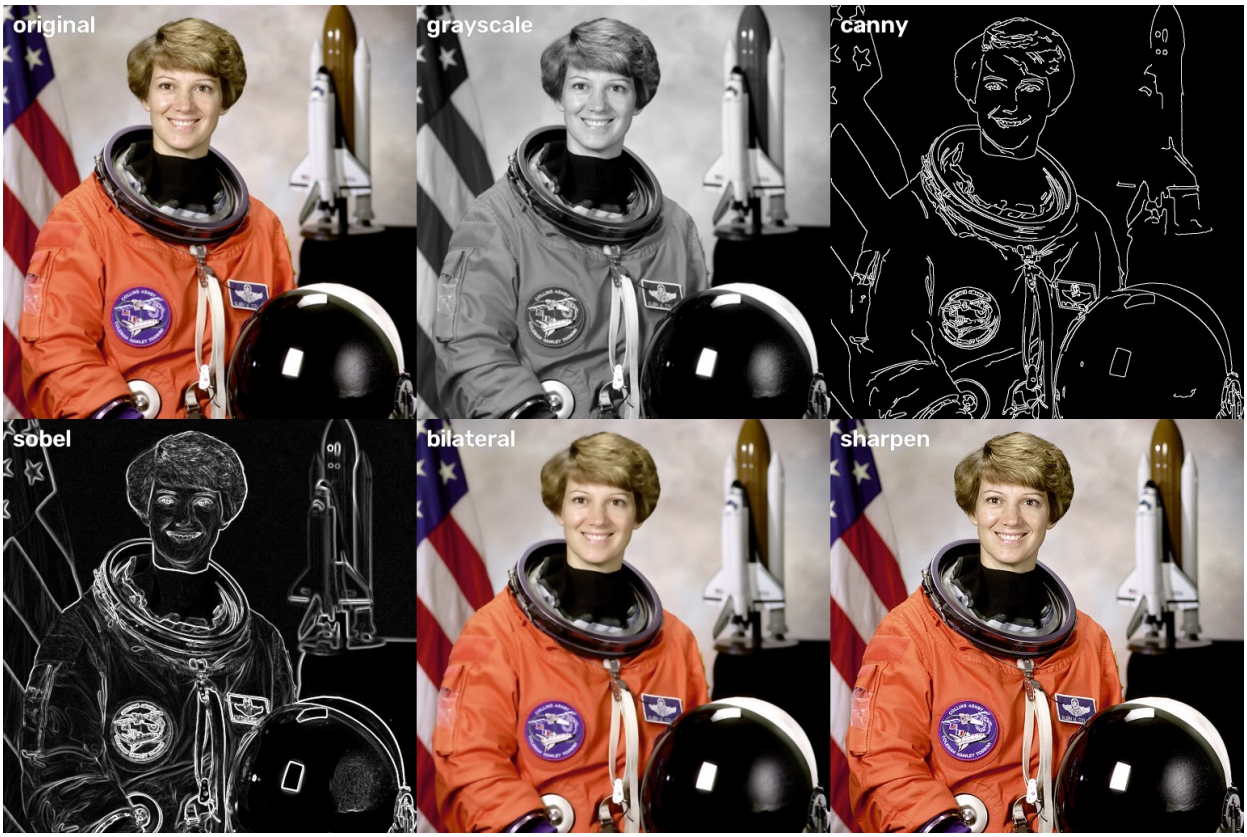


Figure 7: Outputs of the tiled, parallel pipeline for each supported operation

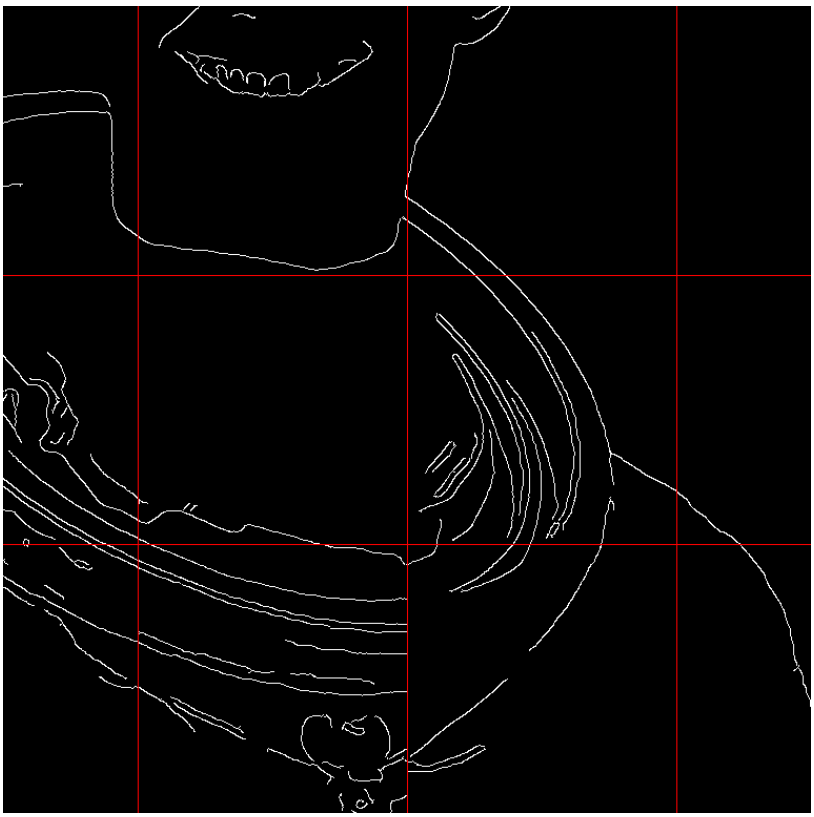


Figure 8: Full-resolution Canny output around tile boundaries (red lines). Edges continue across the joins: the halos remove seams.

Automated tests compare the tiled result with whole-image processing: Gaussian blur, Sobel, sharpen, bilateral and grayscale are **bit-for-bit identical**; Canny agrees on more than 99.97 % of pixels (the end-to-end demo measured 99.98 %).

7.3 Fault-Tolerance Demonstration

While a 256-tile job was running, one of the four workers was killed with SIGKILL, simulating a crashed pod. The tile it was holding stayed in `queue:processing` until its lease expired. The reaper then requeued it, a surviving worker processed it, and the job completed with a correct result and `retries = 1`.

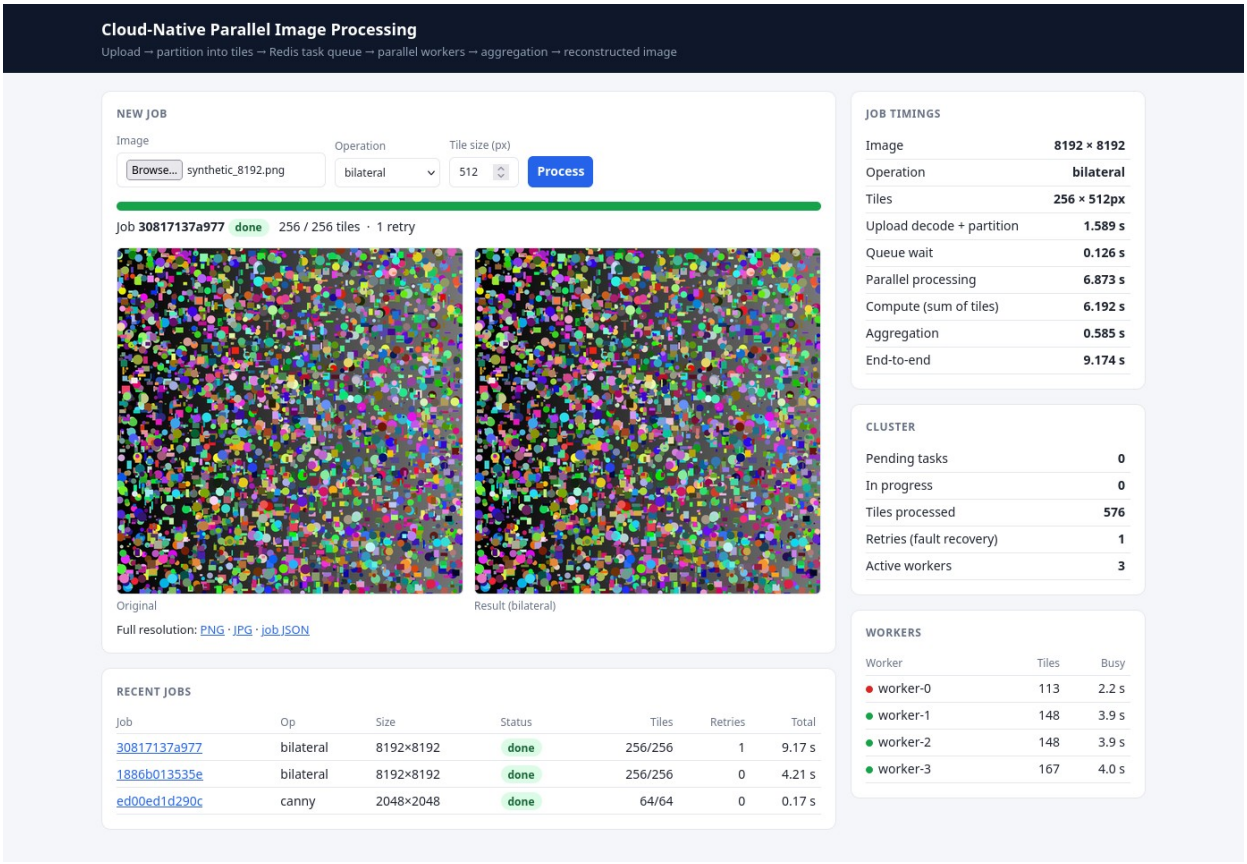


Figure 9: Job completed after a worker was killed: 1 retry, the dead worker shown in red, the other three workers finished the job

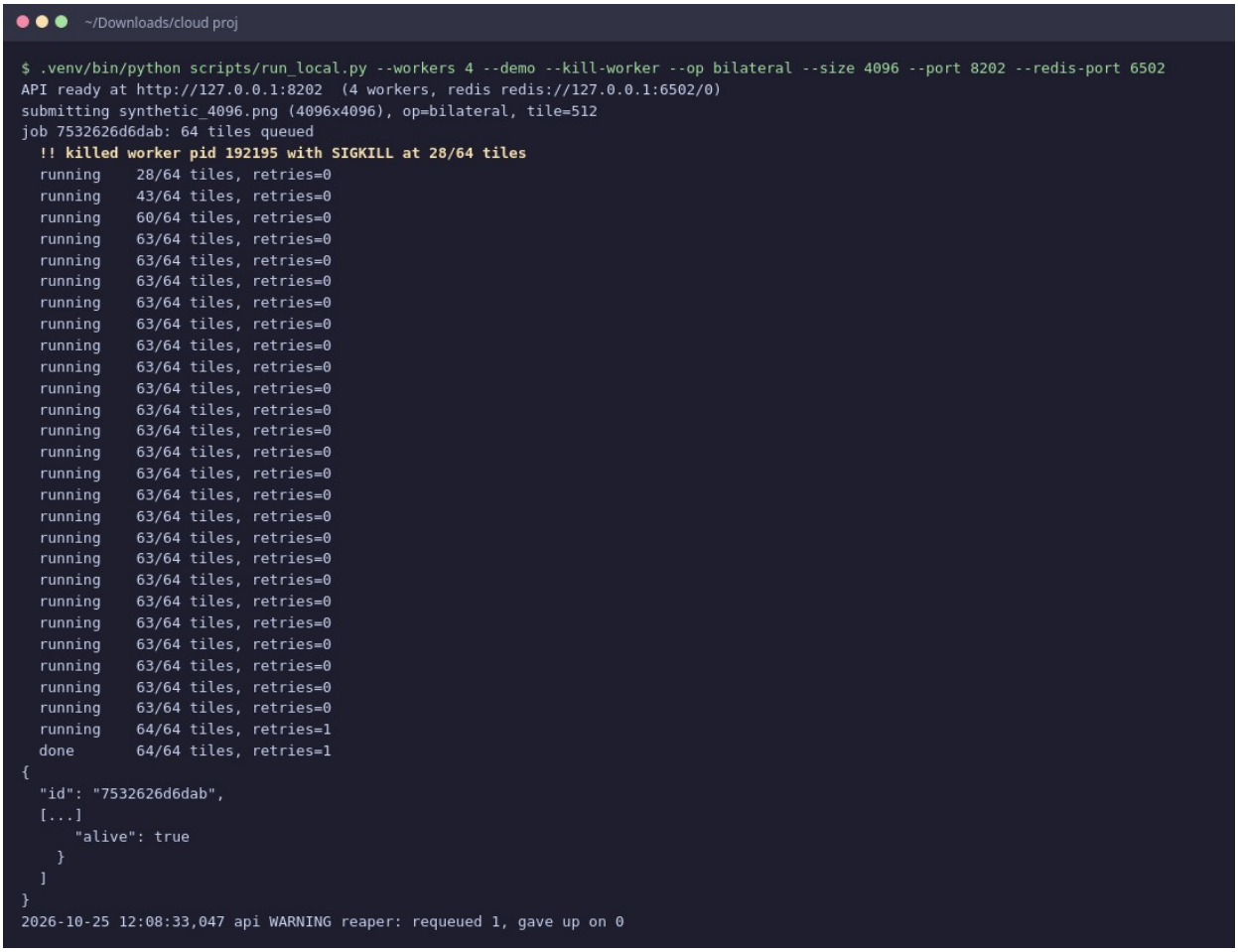


Figure 10: Command-line fault-tolerance demo: worker killed mid-job, job completes, output 100 % identical to the sequential result

7.4 End-to-End Command-Line Demo, API and Metrics

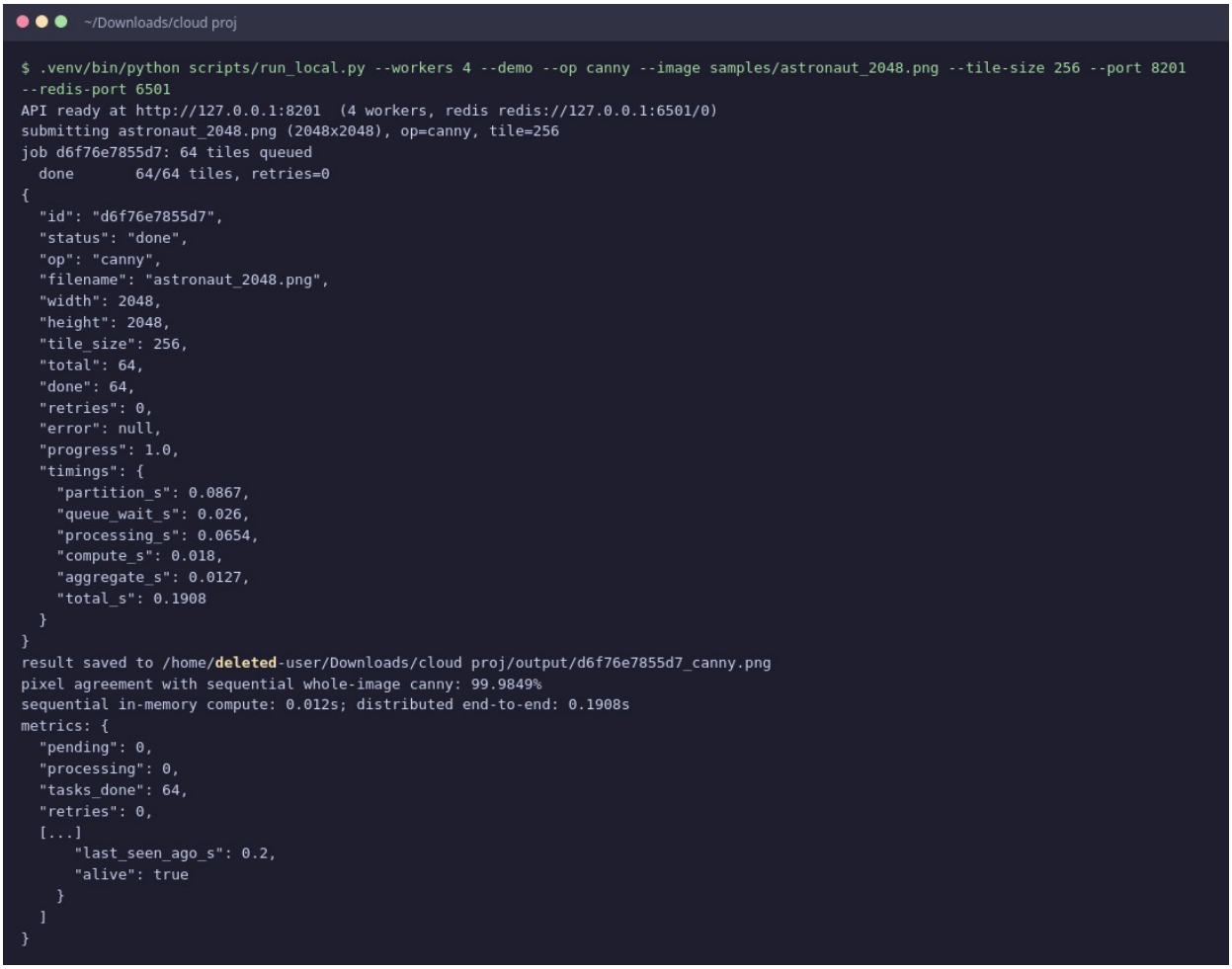


Figure 11: Command-line demo: upload, 64 tiles processed by 4 workers, result verified against sequential Canny

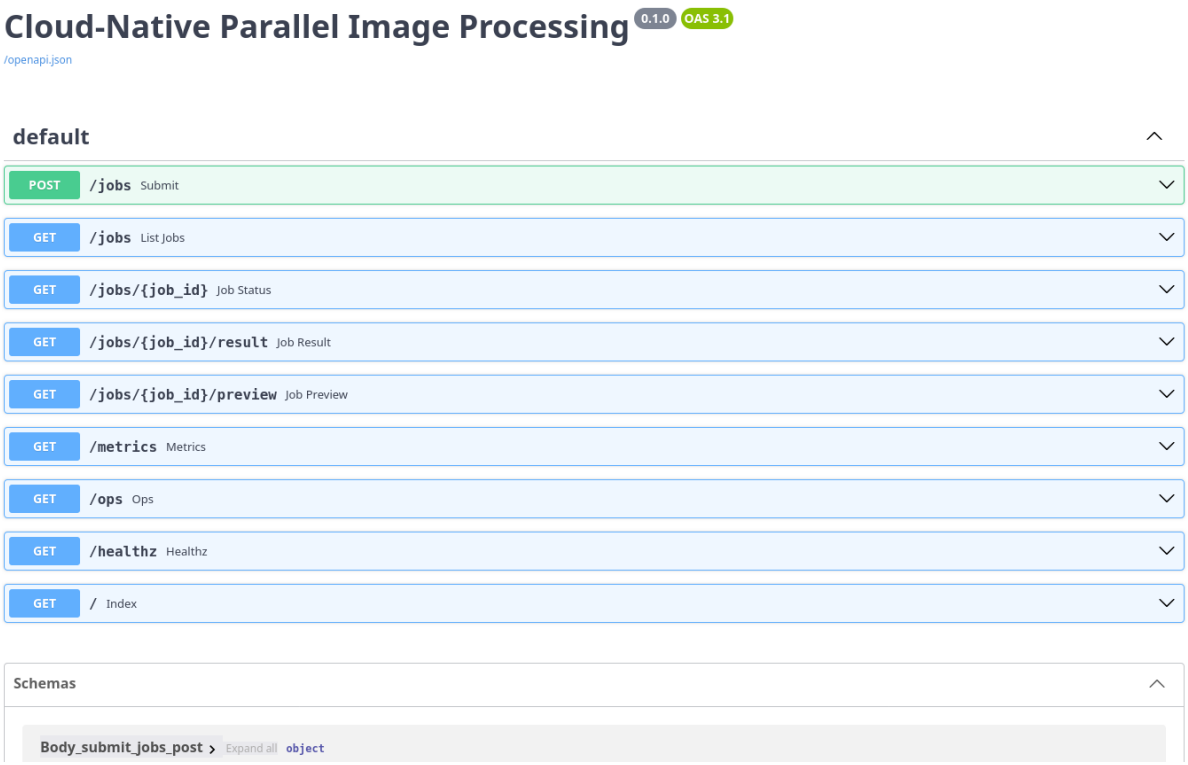


Figure 12: Interactive API documentation generated by FastAPI

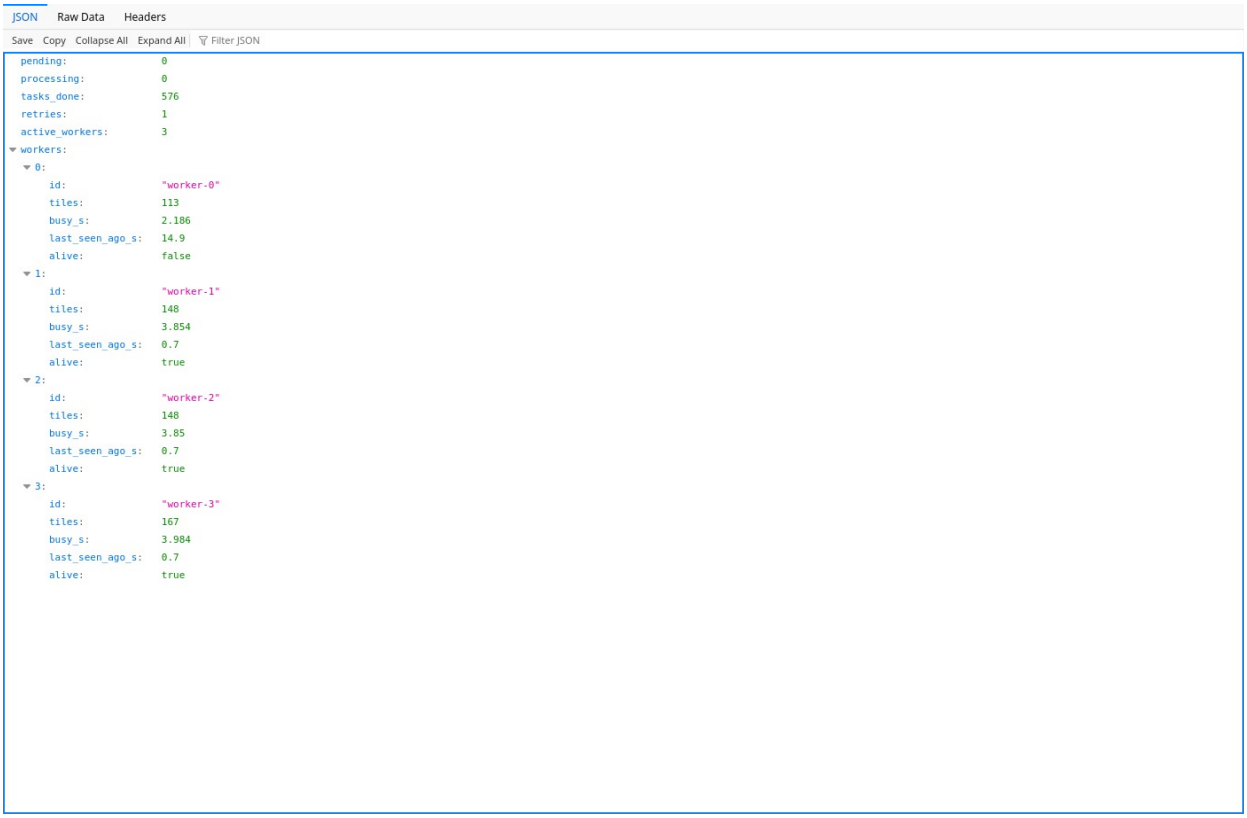


Figure 13: The /metrics endpoint used by the UI and the autoscaler

8. Performance Evaluation

8.1 Experimental Setup

Table 8: Benchmark configuration

Item	Value
Machine	8 physical / 16 logical cores, 23 GB RAM, Linux 6.17
Test image	8192 × 8192 px, 3 channels (192 MB), synthetic shapes + noise (deterministic)
Tile size	512 px → 256 tiles
Operations	bilateral (CPU-heavy), canny and gaussian_blur (lightweight)
Workers	1, 2, 4, 8, 16 processes (compute benchmark); 1, 2, 4, 8 workers (framework)
Method	Median of 3 runs (compute), best of 2 (framework); OpenCV limited to 1 thread per process; CPU and memory sampled every 100 ms with psutil
Metrics	Processing time, speedup S = T_seq / T_par, efficiency E = S / N, CPU cores busy, peak memory

Two experiments were run:

- **Compute-level** (benchmark/bench_local.py): the partition → parallel process → reconstruct engine with N worker processes, without network or Redis. It measures how well the work itself parallelises.
- **Full framework** (benchmark/bench_distributed.py): the real system (HTTP upload to the API, Redis queue, N worker processes, aggregation), timed from upload until the reconstructed image is ready.

8.2 Compute-Level Speedup

Table 9: Sequential vs parallel, 8192^2 image

Operation	Workers	Time (s)	Speedup	Efficiency	Cores busy	Peak mem (MB)	Match
bilateral	sequential	5.323	1.00	100 %	2.2	635	100.000 %
bilateral	1	6.339	0.84	84 %	2.2	1340	100.000 %
bilateral	2	3.410	1.56	78 %	3.2	1558	100.000 %
bilateral	4	1.898	2.80	70 %	5.4	1543	100.000 %
bilateral	8	1.318	4.04	50 %	8.4	1904	100.000 %
bilateral	16	1.271	4.19	26 %	13.2	2264	100.000 %
canny	sequential	0.211	1.00	100 %	3.5	965	100.000 %
canny	1	0.826	0.26	26 %	3.6	1179	99.976 %
canny	2	0.568	0.37	19 %	4.6	1027	99.976 %
canny	4	0.558	0.38	9 %	6.4	1087	99.976 %
canny	8	0.502	0.42	5 %	6.5	1265	99.976 %
canny	16	0.470	0.45	3 %	7.7	1653	99.976 %
gaussian_blur	sequential	0.288	1.00	100 %	2.7	964	100.000 %
gaussian_blur	1	1.339	0.22	22 %	2.9	1440	100.000 %
gaussian_blur	2	0.904	0.32	16 %	4.2	1568	100.000 %

gaussian_blur	4	0.693	0.42	10 %	6.5	1548	100.000 %
gaussian_blur	8	0.639	0.45	6 %	7.4	1925	100.000 %
gaussian_blur	16	0.701	0.41	3 %	7.8	2262	100.000 %

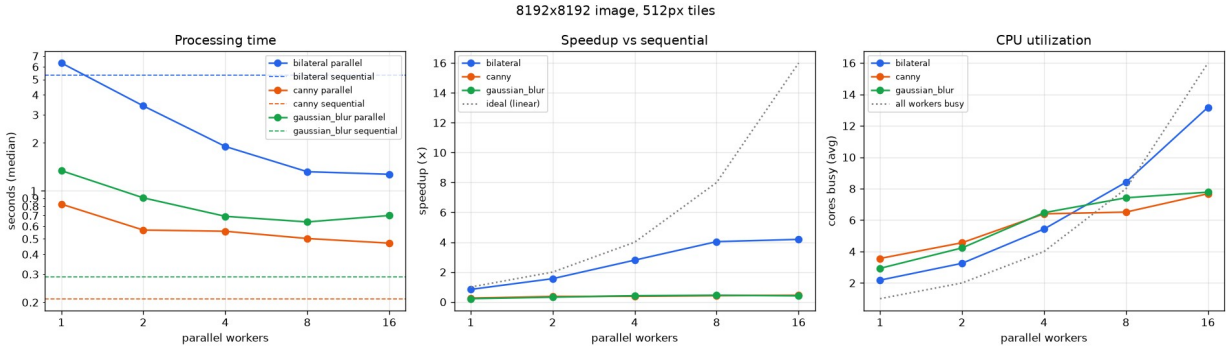


Figure 14: Processing time (log scale), speedup and CPU utilisation versus number of workers

```
~/Downloads/cloud proj

$ .venv/bin/python -m benchmark.bench_local --size 8192 --ops bilateral canny gaussian_blur --workers 1 2 4 8 16
machine: 8 physical / 16 logical cores, 23 GB RAM
image: 8192x8192 (192 MB), tile 512px

[bilateral] sequential: 5.323s  cpu 2.21 cores
[bilateral]  1 workers: 6.339s  speedup  0.84x  efficiency  84%  cpu 2.18 cores  mem 1339.8 MB  match 100.000%
[bilateral]  2 workers: 3.410s  speedup  1.56x  efficiency  78%  cpu 3.25 cores  mem 1558.3 MB  match 100.000%
[bilateral]  4 workers: 1.898s  speedup  2.80x  efficiency  70%  cpu 5.43 cores  mem 1543.3 MB  match 100.000%
[bilateral]  8 workers: 1.318s  speedup  4.04x  efficiency  50%  cpu 8.41 cores  mem 1904.0 MB  match 100.000%
[bilateral] 16 workers: 1.271s  speedup  4.19x  efficiency  26%  cpu 13.18 cores  mem 2264.0 MB  match 100.000%

[canny] sequential: 0.211s  cpu 3.5 cores
[canny]  1 workers: 0.826s  speedup  0.26x  efficiency  26%  cpu 3.56 cores  mem 1179.2 MB  match 99.976%
[canny]  2 workers: 0.568s  speedup  0.37x  efficiency  19%  cpu 4.56 cores  mem 1026.7 MB  match 99.976%
[canny]  4 workers: 0.558s  speedup  0.38x  efficiency   9%  cpu 6.4 cores  mem 1087.0 MB  match 99.976%
[canny]  8 workers: 0.502s  speedup  0.42x  efficiency   5%  cpu 6.51 cores  mem 1264.8 MB  match 99.976%
[canny] 16 workers: 0.470s  speedup  0.45x  efficiency   3%  cpu 7.68 cores  mem 1652.6 MB  match 99.976%

[gaussian_blur] sequential: 0.288s  cpu 2.67 cores
[gaussian_blur]  1 workers: 1.339s  speedup  0.22x  efficiency  22%  cpu 2.93 cores  mem 1439.6 MB  match 100.000%
[gaussian_blur]  2 workers: 0.904s  speedup  0.32x  efficiency  16%  cpu 4.23 cores  mem 1568.1 MB  match 100.000%
[gaussian_blur]  4 workers: 0.692s  speedup  0.42x  efficiency  10%  cpu 6.47 cores  mem 1547.5 MB  match 100.000%
[gaussian_blur]  8 workers: 0.639s  speedup  0.45x  efficiency   6%  cpu 7.42 cores  mem 1924.6 MB  match 100.000%
[gaussian_blur] 16 workers: 0.701s  speedup  0.41x  efficiency   3%  cpu 7.78 cores  mem 2262.0 MB  match 100.000%

wrote benchmark/results/results.csv, results.json, charts.png
```

Figure 15: Benchmark run (terminal output)

8.3 End-to-End Speedup of the Full Framework

Table 10: Full framework, bilateral filter on 8192^2 image

Workers	End-to-end (s)	Speedup	Upload+partition (s)	Parallel phase (s)	Phase speedup	Aggregation (s)
sequential	5.32	1.00	—	5.32	1.00	—
1	8.88	0.60	1.47	6.35	0.84	0.49
2	5.83	0.91	1.45	3.42	1.55	0.46
4	4.55	1.17	1.47	1.98	2.69	0.57
8	3.80	1.40	1.48	1.25	4.27	0.58

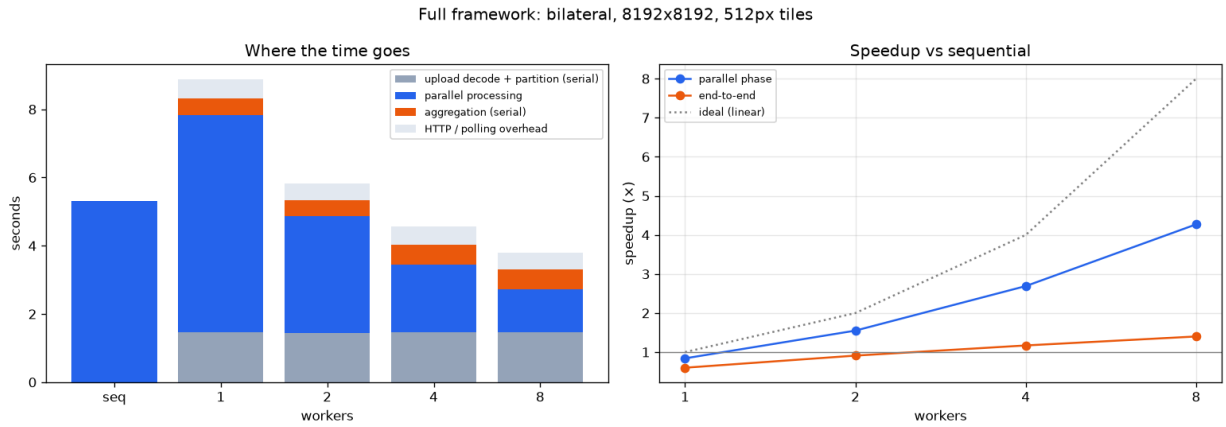


Figure 16: Where the time goes in the full framework, and speedup of the parallel phase versus end-to-end

```
~/Downloads/cloud proj

$ .venv/bin/python -m benchmark.bench_distributed --workers 1 2 4 8 --op bilateral --size 8192
bilateral on 8192x8192: sequential whole-image 5.322s

  1 workers: end-to-end  8.878s (0.60x)  |  parallel phase  6.354s (0.84x)  |  partition 1.4746s, aggregate 0.4933s, 2.27 cores busy
  2 workers: end-to-end  5.830s (0.91x)  |  parallel phase  3.425s (1.55x)  |  partition 1.4502s, aggregate 0.4611s, 2.74 cores busy
  4 workers: end-to-end  4.554s (1.17x)  |  parallel phase  1.979s (2.69x)  |  partition 1.4658s, aggregate 0.5737s, 3.46 cores busy
  8 workers: end-to-end  3.798s (1.40x)  |  parallel phase  1.247s (4.27x)  |  partition 1.4758s, aggregate 0.5806s, 4.43 cores busy

wrote benchmark/results/distributed_bilateral.csv and .json
```

Figure 17: Full-framework benchmark run (terminal output)

8.4 Analysis

CPU-heavy work scales well. The bilateral filter takes 5.32 s sequentially. With 2, 4 and 8 workers the speedup is 1.56×, 2.80× and 4.04×. CPU utilisation rises with the worker count, so the extra workers are actually used. Efficiency falls as N grows (78 % at 2, 50 % at 8) because of the serial partition/stitch steps, the cost of copying tiles to and from worker processes, and memory bandwidth shared by all cores. Going from 8 to 16 workers adds only a little (4.19×): the machine has 8 physical cores, and the second hardware thread of each core shares its execution units.

Lightweight work does not benefit. OpenCV's Canny and Gaussian blur are highly optimised (SIMD) and process the whole 67-megapixel image in about 0.21 s. Sending the tiles to other processes and collecting the results costs more than that, so the parallel versions are slower (speedup below 1). Distributing work only pays off when the computation per byte moved is large enough: the compute-to-communication ratio.

Amdahl's law limits the end-to-end gain. In the full framework the parallel phase sped up 4.27× with 8 workers (6.35 s → 1.25 s), but the end-to-end time improved 1.40× (5.32 s sequential → 3.80 s). Decoding the uploaded PNG and partitioning (1.48 s) and aggregation (0.58 s) are serial, and HTTP transfer and polling add about 0.49 s. Even with infinitely many workers the end-to-end speedup could not exceed about 2.1× for this job. Larger images, heavier operations, or parallelising the partition and aggregation steps would raise this bound.

Resource utilisation. Peak memory grows with the number of workers because every process holds its own copies of the tiles it is working on; with 512 px tiles this stays around 1–2 GB for a 192 MB image. In the full framework Redis holds each job's input and output tiles (about twice the raw image size) until aggregation, after which they are deleted. The Redis deployment is therefore given a 3 GB memory limit (enough for four concurrent 67-megapixel jobs).

Fault tolerance cost. In the fault demonstration the job still finished with an identical result. The only cost was the lease timeout (5 s locally, 10 s on Kubernetes) before the lost tile was retried. A shorter lease recovers faster but risks retrying tiles that are only slow.

9. Testing

The project has 26 automated tests (pytest). They run against an in-memory Redis (fakeredis) with the same Lua scripts, so they need no external services.

Table 11: Automated tests

Area	Tests	What is checked (all passed)
Partitioning and reconstruction tests/test_tiling.py	13	test tiles cover image exactly once; test split and reconstruct roundtrip; test local filters match whole image exactly [gaussian blur]; test local filters match whole image exactly [sobel]; test local filters match whole image exactly [sharpen]; test local filters match whole image exactly [grayscale]; test local filters match whole image exactly [bilateral]; test canny matches whole image; test parallel equals sequential; test png codec roundtrip; test unknown op; test raw tile codec roundtrip [shape0]; test raw tile codec roundtrip [shape1]
Reliable queue (claim, complete, lease expiry, retries) tests/test_store.py	6	test claim complete counts progress; test duplicate completion is ignored; test expired lease is requeued; test heartbeat extends lease; test job fails after max retries; test release does not count as attempt
End-to-end through the HTTP API, fault recovery, autoscaler policy tests/test_pipeline.py	7	test upload process download [canny]; test upload process download [gaussian blur]; test worker crash is recovered; test bad requests; test autoscaler policy; test delete job; test backpressure when redis is full

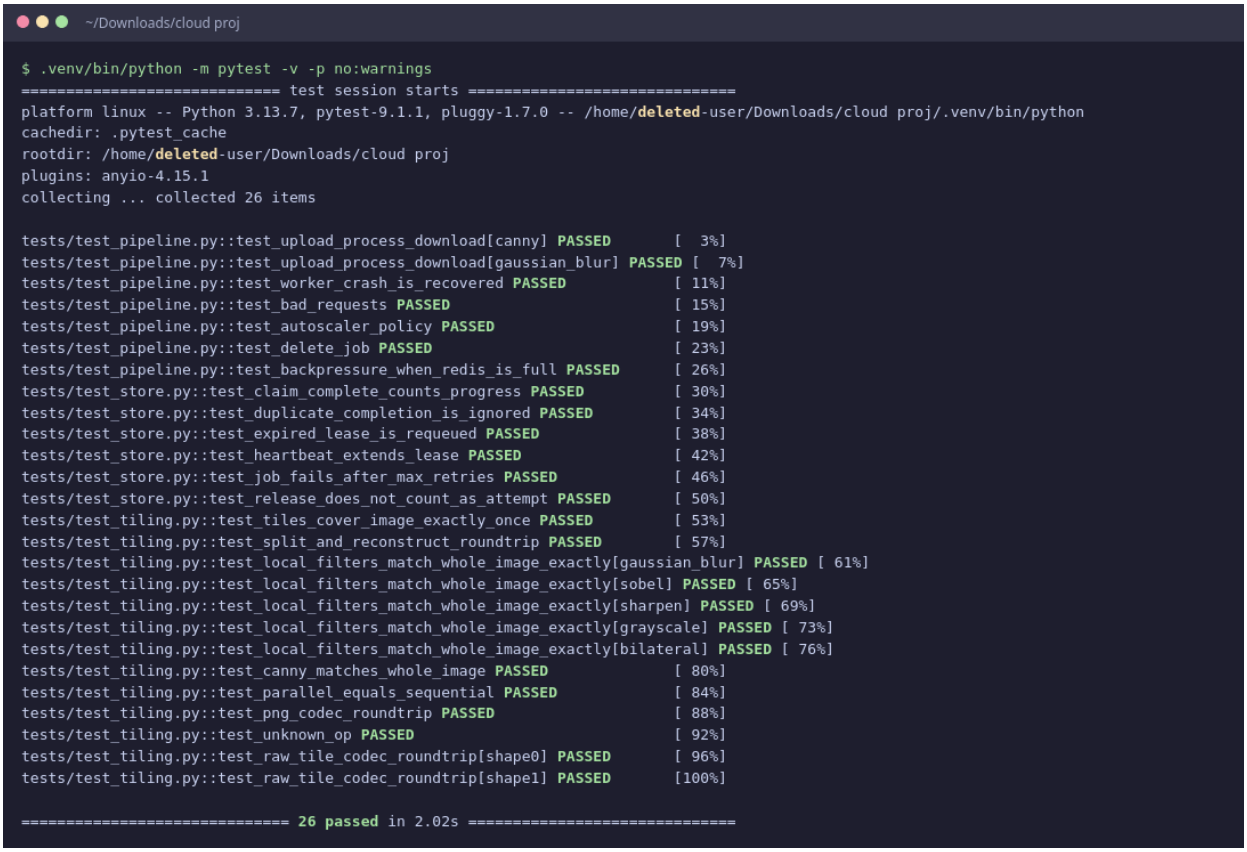


Figure 18: Test run: 26 passed in 2.02s

10. Deployment on Kubernetes and Mapping to AWS

The framework was deployed on a single-node Kubernetes cluster created with Minikube (v1.37, Kubernetes v1.34, Docker driver, 8 CPUs and 8 GB of memory allocated) on the development machine. Each worker pod is limited to 1 CPU, so the number of pods is the degree of parallelism.

10.1 Deployment

```
minikube start --driver=docker --cpus=8 --memory=8g
eval $(minikube docker-env)          # build straight into Minikube's Docker
docker build -t imgframe:latest .
kubectl apply -f k8s/
kubectl -n imgframe get pods -o wide   # redis, api, workers, autoscaler
# Web UI: http://$(minikube ip):30080
```

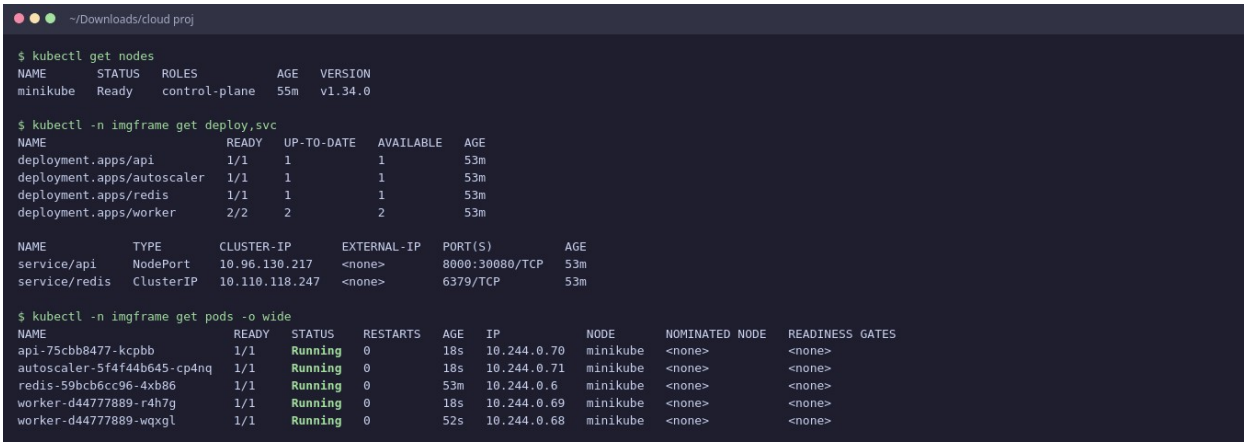


Figure 19: The deployment running on Minikube: Deployments, Services and pods

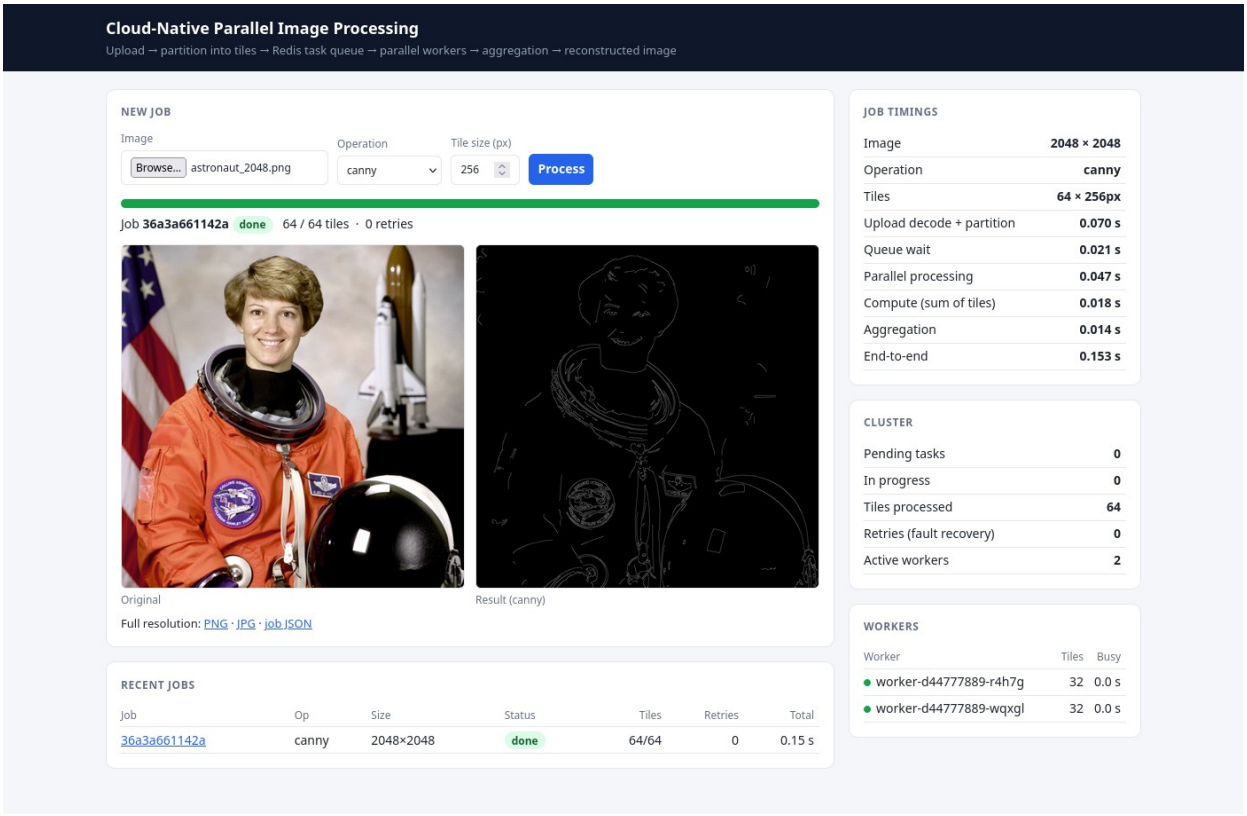


Figure 20: The web UI served by the API pod through the NodePort Service. The workers listed are Kubernetes pods.

10.2 Dynamic Scaling

Four 67-megapixel jobs (1024 tiles in total) were submitted at once with scripts/load_test.py while the worker Deployment was at its minimum of one pod. The autoscaler saw the queue grow and raised the replica count 2 s after the first tiles were queued. Kubernetes had **8 worker pods** ready 3 s after the load started. When the queue emptied the pods stayed for the 30-second cool-down and were then scaled back to one (29 s after the queue emptied).

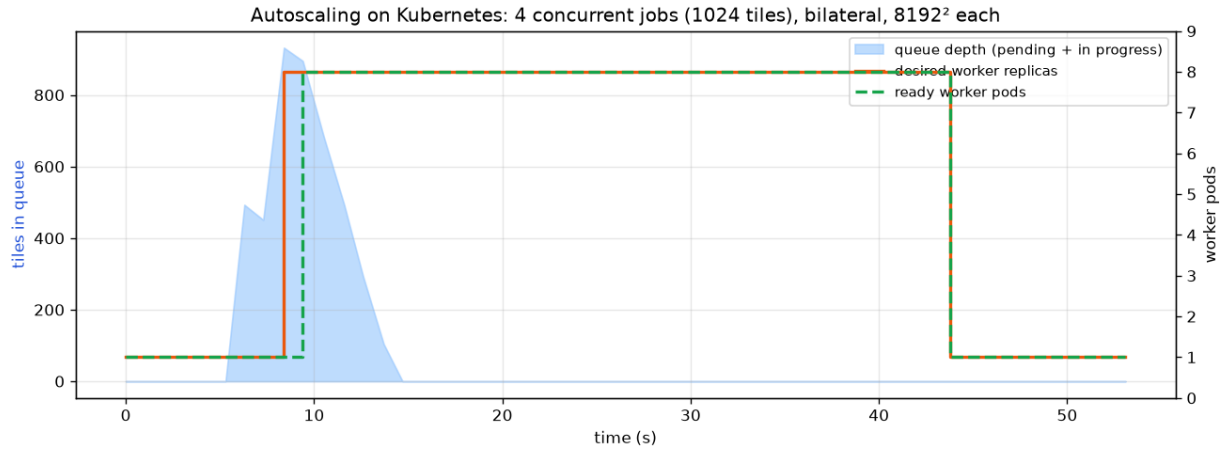


Figure 21: Queue depth and worker pods over time during the autoscaling test

```
~/Downloads/cloud proj

$ kubectl -n imgframe logs deploy/autoscaler | grep scaling
2026-10-24 18:20:19,021 autoscaler INFO autoscaling imgframe/worker between 1 and 8 replicas (16 tasks per worker)
2026-10-24 18:20:49,057 autoscaler INFO queue depth 0: scaling down 2 -> 1
2026-10-24 18:21:07,081 autoscaler INFO queue depth 695: scaling up 1 -> 8
2026-10-24 18:21:43,125 autoscaler INFO queue depth 0: scaling down 8 -> 1
```

Figure 22: Autoscaler decisions (scale up on demand, scale down after the cool-down)

```
~/Downloads/cloud proj

$ python scripts/load_test.py --api http://192.168.49.2:30080 --jobs 4 --op bilateral --image samples/synthetic_8192.png
submitting 4 x bilateral jobs to http://192.168.49.2:30080 ...
queued 1024 tiles in 5.0s: d2afe7de8145, 5efcb6e3d504, 46ea0aadcacb, 3d0a43895360

  time  pending  in-progress  workers  retries  jobs done
  5.0s      940            1        1        0  0/4
  6.0s      908            1        1        0  0/4
  7.0s      746            7        8        0  0/4
  8.0s      563            7        8        0  1/4
  9.1s      381            6        8        0  1/4
 10.1s      180            7        8        0  2/4
 11.1s        0            0        8        0  3/4
 12.1s        0            0        8        0  4/4

all jobs finished in 12.1s
  d2afe7de8145: done, 256 tiles, 0 retries, end-to-end 8.0894s
  5efcb6e3d504: done, 256 tiles, 0 retries, end-to-end 8.5595s
  46ea0aadcacb: done, 256 tiles, 0 retries, end-to-end 7.3666s
  3d0a43895360: done, 256 tiles, 0 retries, end-to-end 6.7949s

$ kubectl -n imgframe get pods -l app=worker   # at peak
NAME                      READY   STATUS    RESTARTS   AGE
worker-d44777889-686qh   1/1     Running   0          1s
worker-d44777889-9krdx   1/1     Running   0          1s
worker-d44777889-cnqpl   1/1     Running   0          1s
worker-d44777889-lkh8f   1/1     Running   0          1s
worker-d44777889-lswcr   1/1     Running   0          1s
worker-d44777889-rngfw   1/1     Running   0          1s
worker-d44777889-vlgrq   1/1     Running   0          1s
worker-d44777889-wqxgl   1/1     Running   0          89s
```

Figure 23: Load test output and the worker pods at peak

10.3 Fault Tolerance on Kubernetes

Two kinds of failure were injected while jobs were running. In the first, a worker's container was killed with SIGKILL on the node, as when a process crashes or is OOM-killed. Its tile lease expired and the reaper requeued the tile, so the load test reports a retry. Kubernetes restarted the container (RESTARTS = 1), and all jobs completed. In the second, a worker pod was deleted with kubectl delete pod, as happens during scale-down, eviction or a rolling update. The pod received SIGTERM, its worker handed the in-flight tile straight back to the queue without waiting for the lease, and the Deployment created a replacement pod immediately.

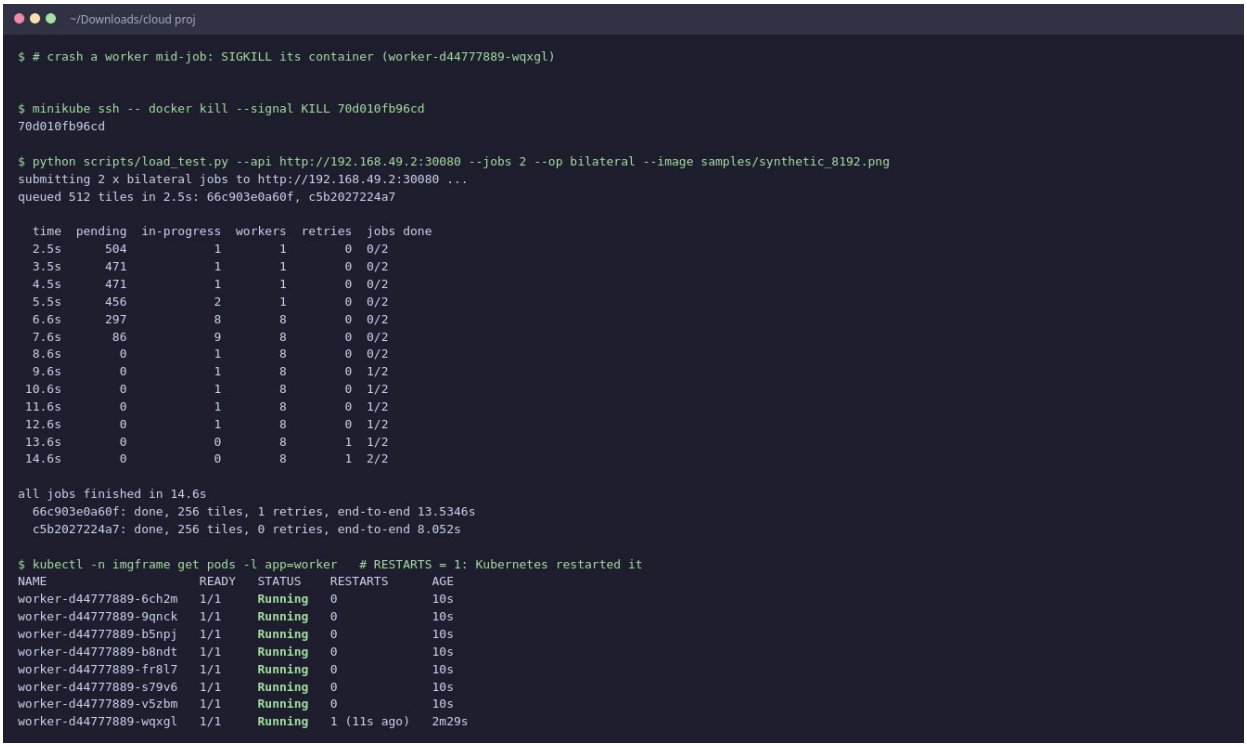


Figure 24: Worker container killed mid-job: tile retried, container restarted by Kubernetes

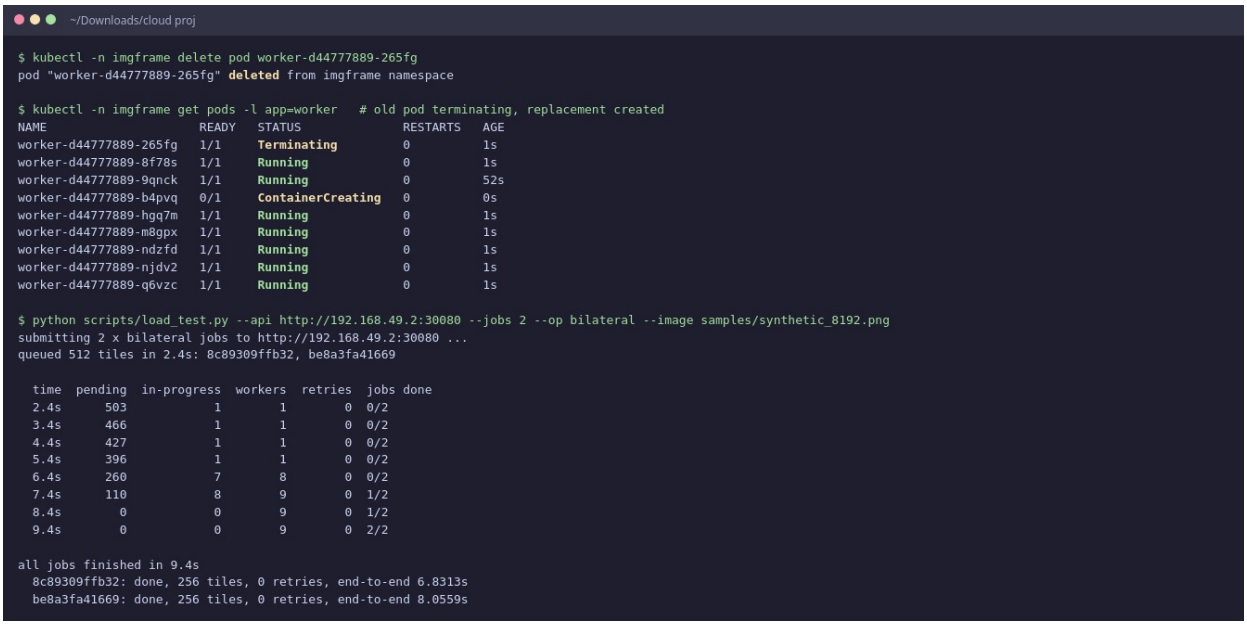


Figure 25: Worker pod deleted mid-job: graceful hand-back and an immediate replacement pod

10.4 Performance on Kubernetes

The full-framework benchmark was repeated against the cluster: the autoscaler was paused and the worker Deployment scaled to 1, 2, 4 and 8 pods by hand. Images were uploaded through the NodePort Service. The sequential baseline is the same operation on one core of the host.

Table 12: Kubernetes results, bilateral filter on an 8192^2 image

Worker pods	End-to-end (s)	Speedup	Upload+partition (s)	Parallel phase (s)	Phase speedup	Aggregation (s)

sequential	5.14	1.00	—	5.14	1.00	—
1	8.71	0.59	1.45	6.25	0.82	0.44
2	5.94	0.87	1.43	3.42	1.51	0.54
4	4.40	1.17	1.42	2.00	2.57	0.45
8	3.64	1.43	1.44	1.26	4.10	0.42

With 8 pods the parallel phase ran 4.10× faster than sequential and the whole job 1.43× faster (5.14 s → 3.64 s). The pattern matches the local measurements in Section 8: near-linear gains in the parallel phase, bounded overall by the serial upload, partition and aggregation steps.

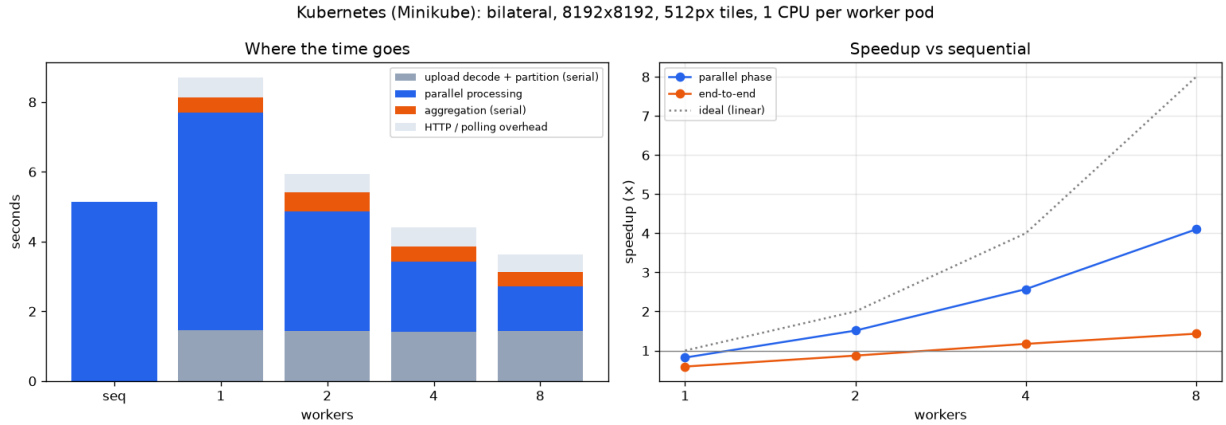


Figure 26: Kubernetes: where the time goes, and speedup versus worker pods

Lessons from deployment. The first runs on the cluster exposed a problem that the local tests had not: every finished job kept its full-resolution result (about 200 MB of raw pixels for an 8192^2 image) in Redis for an hour. After a dozen demonstration jobs Redis reached its memory limit and refused writes. The workers' status updates then failed, crashing the workers into CrashLoopBackOff, and the API pod was OOM-killed while several large uploads waited. Five changes fixed it, and the measurements in this section were taken afterwards: (1) admission control: the API estimates a job's Redis footprint and answers HTTP 503 with Retry-After when the budget would be exceeded; (2) at most two uploads are partitioned at once, bounding API memory; (3) results expire after 15 minutes and DELETE /jobs/{id} frees them immediately; (4) workers survive Redis errors in their status loop; (5) clients retry on 503. Back-pressure of this kind is standard practice for cloud services and is the behaviour expected from a managed queue.

10.5 Docker Compose

The same image also runs with Docker Compose (Redis, API and a scalable number of worker containers). Killing a worker container mid-job with docker kill gives the same lease-based recovery.

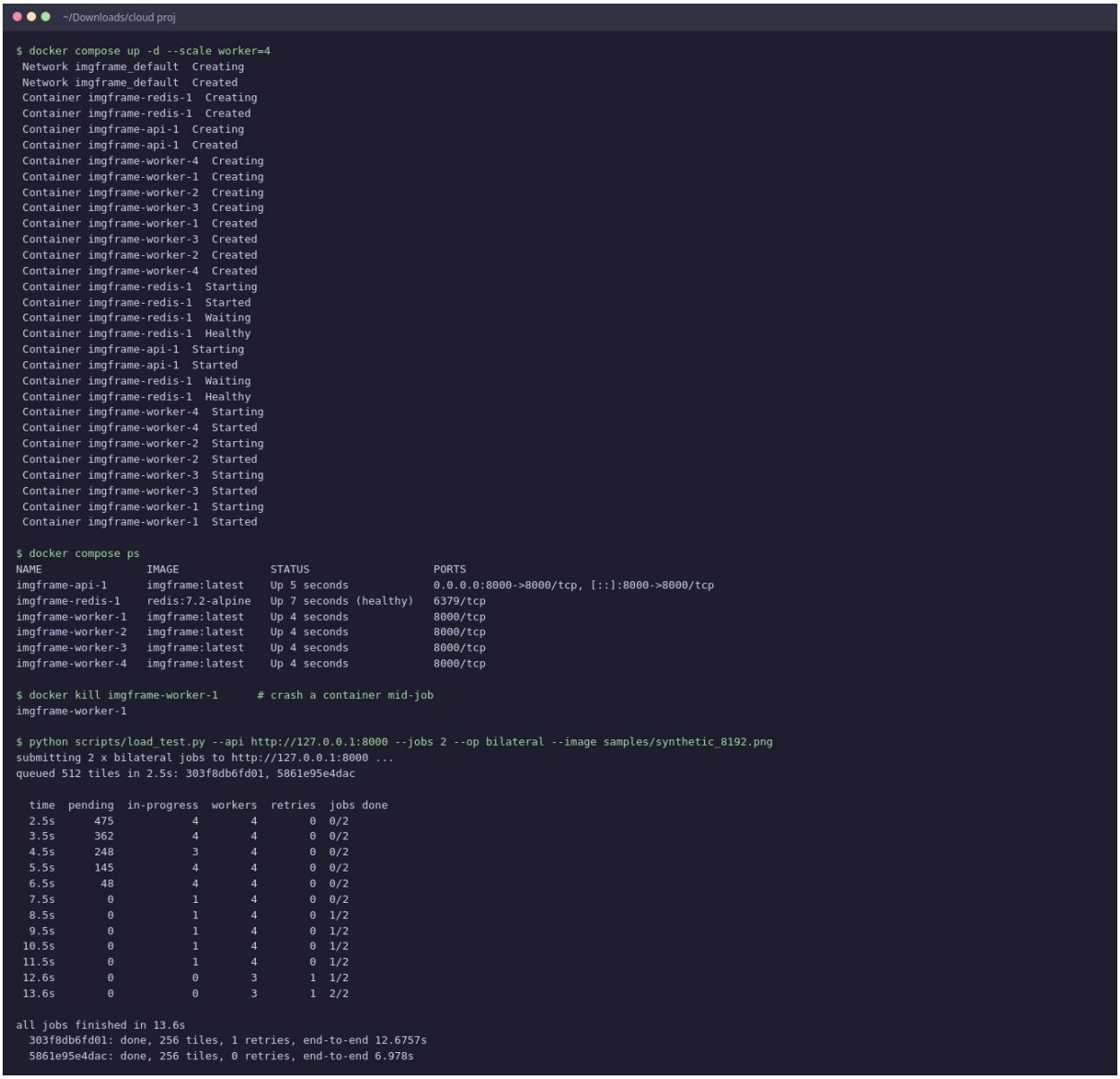


Figure 27: Docker Compose: four worker containers, one killed mid-job, jobs still complete

10.6 Mapping to AWS Services

The same design maps directly onto managed AWS services covered in the course:

Table 13: Mapping to AWS

Component	Local / Minikube	AWS equivalent
Container registry	Minikube Docker daemon	Amazon ECR
Orchestration	Minikube	Amazon EKS (or ECS on Fargate)
Task queue + store	Redis Deployment	Amazon ElastiCache for Redis (Multi-AZ); large images in Amazon S3
Worker scaling	Queue autoscaler / HPA / KEDA	KEDA or HPA on EKS + Cluster Autoscaler / Karpenter for nodes
Ingress	NodePort 30080	Application Load Balancer (AWS Load Balancer Controller)
Permissions	Kubernetes RBAC	RBAC + IAM Roles for Service Accounts (IRSA)

Monitoring	/metrics endpoint	Amazon CloudWatch (Container Insights)
Batch alternative	—	AWS Lambda per tile triggered by SQS for bursty workloads

11. Limitations and Future Work

- **Serial stages.** Upload decoding, partitioning and aggregation run in one process and cap the end-to-end speedup. Workers could read their tile regions directly from object storage (S3) and write results in place, removing both serial steps.
- **Redis as a data store.** Tiles pass through Redis memory, which limits image size to the Redis memory limit. Storing pixels in S3 or a shared volume and keeping only metadata in Redis would remove this limit.
- **Single Redis instance.** Redis is a single point of failure in this deployment. A managed replicated service (ElastiCache Multi-AZ) or Redis Sentinel would address this.
- **Canny is not exactly tile-local.** Its hysteresis stage can follow edges further than any finite halo, so a tiny fraction of pixels can differ.
- **Predictive scaling.** Scaling reacts to queue length. Scaling from the size of incoming jobs, or pre-warming pods, would reduce start-up latency [4].
- **GPU workers.** Heavy operations could use CUDA-enabled OpenCV on GPU node pools.

12. Conclusion

The project delivers a working cloud-native framework for parallel image processing. A large image is partitioned into halo tiles, scheduled through an atomic Redis-based queue, processed by stateless parallel workers and reconstructed into an output identical to whole-image processing. The framework tolerates worker failures through leases, heartbeats and idempotent completion, scales workers from queue length, and is packaged for Docker and Kubernetes. Measurements show a 4.04× speedup on 8 workers for CPU-heavy work and clearly show the two classic limits of parallel computing: the compute-to-communication ratio (light operations do not benefit) and Amdahl's law (serial stages bound the end-to-end gain). These results show which workloads benefit from cloud-native parallelism and where to optimise next.

References

[1] Towards an Adaptive Runtime System for Cloud-Native HPC, 2026. arXiv:2603.14630.

[2] An Elastic Job Scheduler for HPC Applications on the Cloud, SC25, 2025. IEEE Xplore 11358356.

[3] Serverless Data Pipeline Architecture Supporting Distributed Machine Learning on Fog Devices, Computer Communications, 2026.

[4] A Kubernetes-Based Scheme for Efficient Resource Allocation in Containerized Workflow, Future Generation Computer Systems, 2025.

[5] Running Cloud-Native Workloads on HPC with High-Performance Kubernetes, 2024. arXiv:2409.16919.

[6] G. M. Amdahl, Validity of the single processor approach to achieving large scale computing capabilities, AFIPS Conference Proceedings, 1967.

[7] J. Canny, A Computational Approach to Edge Detection, IEEE TPAMI, 1986.

[8] C. Tomasi and R. Manduchi, Bilateral Filtering for Gray and Color Images, ICCV, 1998.

[9] OpenCV documentation, https://docs.opencv.org/4.x/

[10] Redis documentation: Lua scripting and the reliable queue pattern, https://redis.io/docs/

[11] Kubernetes documentation: Deployments, RBAC, Horizontal Pod Autoscaling, https://kubernetes.io/docs/

[12] FastAPI documentation, https://fastapi.tiangolo.com/

Appendix A: Project Structure

```
cloud proj/
├── imgframe/         core package
│   ├── config.py     settings from environment variables
│   ├── ops.py        OpenCV operations + halo sizes
│   ├── tiling.py     plan/split/reconstruct, tile codec
│   ├── store.py      Redis data model + reliable queue (Lua)
│   ├── pipeline.py   partition / process / assemble stages
│   └── sample.py     deterministic synthetic test images
├── api/              FastAPI service + web UI (index.html)
├── worker/           worker process
├── autoscaler/       queue-length autoscaler
├── benchmark/        bench_local, bench_distributed, plots, results/
├── k8s/              Kubernetes manifests (+ optional HPA, KEDA)
├── scripts/          run_local, load_test, kill_worker, install_k8s_tools,
│                     build_redis, capture_screenshots, capture_k8s
├── tests/            pytest suite
├── docs/             report, guide, screenshots, logs
├── Dockerfile, docker-compose.yml, requirements*.txt, README.md
```

Appendix B: Configuration and API Reference

Table 14: Configuration

Variable	Default	Meaning
REDIS_URL	redis:// localhost:6379/0	Redis connection
TILE_SIZE	512	Default tile edge in pixels
LEASE_SECONDS	15 (10 on Kubernetes)	Time a claimed tile may go without a heartbeat
HEARTBEAT_SECONDS	3 (2 on Kubernetes)	Lease renewal interval
MAX_RETRIES	3	Attempts per tile before the job fails
WORKER_PROCS	1	Process-pool slots per worker
OPENCV_THREADS	1	OpenCV internal threads per process

CHAOS_CRASH_RATE	0	Probability that a worker crashes mid-tile (testing)
RESULT_TTL_SECONDS	900	How long a finished image is kept
REDIS_MEMORY_BUDGET	0.9	Fraction of Redis maxmemory usable before new jobs get 503
MAX_CONCURRENT_UPLOADS	2	Uploads partitioned at the same time (bounds API memory)
MIN/MAX_WORKERS, TASKS_PER_WORKER	1 / 8, 16	Autoscaler policy